Implementation of a Configurable PWM Generator

Computer Architecture Project

Team: Matei David, Atudosiei Andrei Cristian, Vrabie Tudor

Contents

1 Introduction

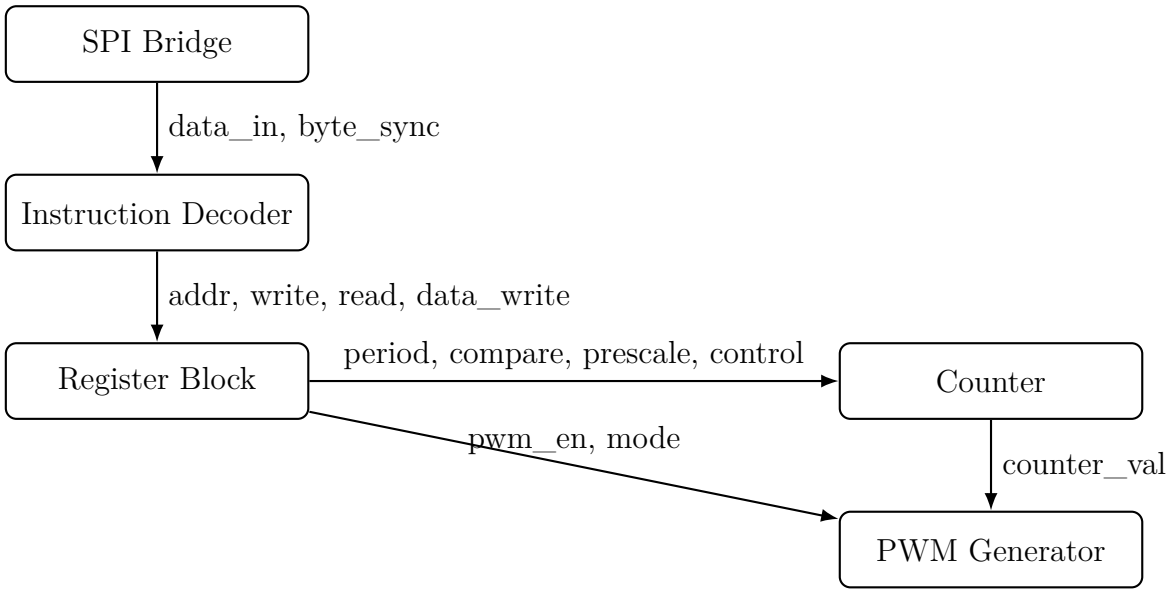
PWM (Pulse Width Modulation) signal generators are essential components in many embedded systems, being used for precise control of devices such as LEDs, electric motors, and power electronics. In modern microcontroller architectures, these generators are typically implemented as programmable peripherals, configurable through internal registers and serial communication interfaces.

In this project, we implemented a complete hardware module capable of generating a configurable PWM signal using the Verilog hardware description language. The peripheral allows the configuration of the main PWM parameters — such as period, duty cycle, alignment mode, and counting direction — through an SPI communication interface, as well as the activation, deactivation, and monitoring of the internal counter.

The architecture of the module consists of five main components: the SPI bridge responsible for receiving and transmitting data, the instruction decoder which interprets byte-level commands, the register block which stores the peripheral configuration, the `counter` module which performs the actual counting operation, and the `pwm_gen` module which generates the PWM output signal. The implementation of each component, along with the specific design decisions taken throughout the project, is presented in the following sections.

2 Architecture Overview

The PWM generator peripheral was designed as a modular hardware system composed of five interconnected components, each responsible for a distinct stage of data processing and signal generation. The architecture follows a clear separation of responsibilities, ensuring synchronous operation, predictable data flow, and ease of verification.



2.1 Module Interactions

The communication flow across components follows a strict pipeline:

1. **SPI Bridge** receives serial data from the external master and converts it into 8-bit parallel words. It also asserts a `byte_sync` signal to indicate the completion of each byte transfer.

2. **Instruction Decoder** interprets these bytes as structured commands. Based on the first byte, it determines whether the operation is a read or write, identifies the target register, and decides whether the high or low byte of a 16-bit register is being accessed.

3. **Register Block** stores all configuration parameters of the peripheral: period, compare values, prescaler settings, enable bits, and internal modes. It exposes these parameters continuously to the counter and PWM generator.

4. **Counter Module** implements the time base of the PWM generator. It increments or decrements based on the configuration, handles prescaling, detects overflow and underflow, and outputs the current counter value.

5. **PWM Generator** produces the final PWM output based on the counter value and compare thresholds. It supports left-aligned, right-aligned, and dual-compare modes, making the output waveform fully configurable.

2.2 Signal Flow Between Components

The following summarizes the key signals exchanged among modules:

- **From SPI Bridge -> Instruction Decoder:**
  - `data_in[7:0]`: parallel byte received from MOSI
  - `byte_sync`: indicates end of an 8-bit transfer
- **From Instruction Decoder -> Register Block:**
  - `addr[5:0]`: target register address
  - `write`, `read`: operation type
  - `data_write[7:0]`: byte to be written into a register
- **From Register Block -> Counter:**
  - `period`, `compare1`, `compare2`
  - `prescale`, `upnotdown`, `counter_reset`, `counter_en`

- **From Counter -> PWM Generator:**
  - `counter_val`: current counter state
  - overflow/underflow signals
- **From Register Block -> PWM Generator:**
  - `functions`: waveform mode configuration
  - `pwm_en`: enables PWM generation

2.3 Design Considerations

Throughout development, several design decisions were made to ensure correctness and reliability:

- **Synchronous operation...**: All modules operate on the same clock domain, avoiding clock-crossing issues and simplifying verification.
- **Full decoupling of control and datapath...**: The SPI bridge and decoder handle all communication logic, allowing the counter and PWM generator to remain purely functional blocks.
- **Atomic configuration updates...**: Register values affecting PWM behavior are applied only at counter overflow or during idle states, preventing partial-update glitches.
- **Scalability...**: The architecture allows extending the register map or adding additional PWM channels with minimal structural modification.

The resulting system is robust, modular, and closely resembles the internal design of real microcontroller peripherals used in embedded platforms.